



MIRADOR 360°

(DAS) Documento Arquitectura de Software
Versión 1.0

Identificación de Documento

Identificación	Grupo 2
Proyecto	Mirador 360°
Versión	1.1
Documento mantenido por	Grupo 2
Fecha de última revisión	-
Fecha de próxima revisión	-
Documento aprobado por	
Fecha de última aprobación	

Historia de Revisiones

Fecha	Versión	Descripción	Autor
09/04/2026	1.1	Levantamiento de requerimientos	Grupo 2

Tabla de Contenidos

1. INTRODUCCIÓN	4
1.1. CONTEXTO DEL PROBLEMA	4
1.2. PROPÓSITO	4
1.3. ÁMBITO	4
1.4. DEFINICIONES, ACRÓNIMOS Y ABREVIACIONES	4
1.5. RESUMEN EJECUTIVO	4
1.6. ARQUITECTURA DEL SISTEMA	4
2. VISIÓN DEL SISTEMA	4
2.1. DESCRIPCIÓN GENERAL DEL SISTEMA	4
2.2. OBJETIVOS DEL SISTEMA	4
2.3. REQUERIMIENTOS FUNCIONALES	4
2.4. REQUERIMIENTOS NO FUNCIONALES	5
2.5. SUPUESTOS Y DEPENDENCIAS	5
3. ESTILOS Y PATRONES ARQUITECTÓNICOS	5
3.2. JUSTIFICACIÓN DEL ESTILO SEGÚN EL CONTEXTO DEL SISTEMA	5
4. MODELO 4 +1 Y VISTAS ARQUITECTÓNICAS	5
4.1. VISTA DE ESCENARIO	5
4.1.1. <i>Propósito</i>	5
4.1.2. <i>Actores</i>	5
4.1.3. <i>Diagrama general de casos de uso</i>	5
4.1.4. <i>Diagrama de casos de uso específicos</i>	5
4.1.6. <i>Especificación de casos de uso</i>	5
4.2. VISTA LÓGICA	6
4.2.1. <i>Propósito</i>	6
4.2.2. <i>Diagrama de clases</i>	6
4.2.3. <i>Descripción diagrama de clases</i>	6
4.3. VISTA DE IMPLEMENTACIÓN/DESARROLLO	7
4.3.1. <i>Propósito</i>	7
4.3.2. <i>Diagrama de componente</i>	7
4.3.3. <i>Descripción diagrama de componente</i>	7
4.3.4. <i>Diagrama de paquete</i>	7
4.3.5. <i>Descripción diagrama de paquete</i>	7
4.4. VISTA DE PROCESOS	7
4.4.1. <i>PROPÓSITO</i>	7
4.4.2. <i>DIAGRAMA DE ACTIVIDAD</i>	7
4.4.3. <i>DESCRIPCIÓN DIAGRAMA DE ACTIVIDAD</i>	7
4.5. VISTA FÍSICA	7
4.5.1. <i>Propósito</i>	7
4.5.2. <i>Diagrama de despliegue</i>	7
4.5.3. <i>Descripción diagrama de despliegue</i>	7
5. REQUISITOS DE CALIDAD	7

5.1. PROPÓSITO	7
5.3. <i>Reglas y criterios de evaluación de calidad</i>	7
6. PRINCIPIOS DE DISEÑO APLICADOS	8
6.1. <i>Propósito</i>	8
6.2. PRINCIPIOS DE DISEÑO (POR EJEMPLO: ABSTRACCIÓN, ACOPLAMIENTO, COHESIÓN, ENCAPSULAMIENTO, MODULARIDAD)	8
7. PROTOTIPO	8
7.1. PROPÓSITO	8
7.2. MOCKUPS (IMÁGENES CON UNA BREVE DESCRIPCIÓN)	8
7.3. JUSTIFICAR HERRAMIENTAS DE PROTOTIPADO	8
8. EVALUACIÓN DE CALIDAD HEURÍSTICA DE NIELSEN	8
8.1. PROPÓSITO	8
8.2. LISTA DE VERIFICACIÓN	8
8.3. ANÁLISIS Y MÉTRICAS DE RESULTADOS	8
9. CONTROL DE VERSIONES	8
9.1. PROPÓSITO	8
9.2. CONTROL DE VERSIÓN UTILIZADO (JUSTIFICAR EL TIPO DE CONTROL DE VERSIÓN UTILIZAD (FECHA, SEMÁNTICA O SECUENCIAL)	8
9.3. JUSTIFICAR HERRAMIENTAS DE VERSIONAMIENTO	8
7. CONCLUSIONES	8
8. BIBLIOGRAFÍA	8
9. ANEXOS	8
9.1. CARTA GANTT	8

1. INTRODUCCIÓN

1.1. Contexto del Problema

Actualmente nos centraremos en el edificio “El Mirador” el cual se encuentra en un escenario desfavorable para su administración por la mala gestión de cobros y transparencia de este mismo ámbito. Presenta varios problemas importantes, principalmente todo el papeleo se realiza a mano, sin programas ni documentos de acceso online, lo que produce errores humanos, atraso con las cuotas, lo que repercute en una mala gestión del mantenimiento y transparencia en las cuentas de este edificio.

1.2. Propósito

En este documento se dejará por escrito y de forma detallada cómo va a estar armado e implementado este nuevo sistema. Desde el equipo a cargo de la programación hasta el usuario final, dando a entender el funcionamiento y lo que se hará durante el proyecto.

1.3. Ámbito

Mirador 360 consiste en la creación e implementación de una página web tipo responsive para que cada actor del edificio tenga acceso a ella, otorgando los permisos correspondientes a cada uno según la necesidad del problema. En la página estará disponible toda la información correspondiente y de utilidad para llevar una cuenta clara de los ingresos y egresos de dinero, además de una sección de reclamos y sugerencias/solicitudes con la mayor transparencia posible.

1.4. Definiciones, acrónimos y abreviaciones

ACRÓNIMO	DESCRIPCIÓN
<i>RESPONSIVE</i>	Diseño de desarrollo web donde el sitio se adapta a cualquier tamaño de pantalla, tablet, computadora, móvil, etc.
<i>MVC</i>	Modelo-Vista-Controlador, tipo de patrón de arquitectura de software que separa una aplicación en 3 partes, mejora la organización y mantenimiento del código
<i>Base de Datos o BD</i>	Es donde se almacena toda la información correspondiente al edificio, nombres, cobros, solicitudes, errores

1.5. Resumen ejecutivo

Básicamente, se busca crear e implementar un sistema automático que reemplace al actual para evitar los problemas que se han generado por el deficiente manejo del sistema de cobro. La idea es que la gente pague a tiempo, las cuentas claras para ambas partes y que la administración no pierda la paciencia y el tiempo cobrando cada que ve un moroso, mejorando así la convivencia del edificio.

1.6. Arquitectura del sistema

Trabajaremos con una arquitectura basada en el patrón MVC (Modelo-Vista-Controlador). El cual nos permite separar en bloques la información que queremos que se muestre a los usuarios finales, para que finalmente en la pantalla de usuario se muestre solo la información relevante para ellos. Además mejora el mantenimiento y corrección de errores del sistema.

2. VISIÓN DEL SISTEMA

2.1. Descripción general del sistema

El sistema será una página web en donde cada usuario podrá ingresar con su cuenta previamente creada, usuario-contraseña. Dentro del sistema los usuarios tendrán acceso a su información personal. Por otro lado la administración tendrá acceso especial para ver los informes detallados del dinero que entra y sale, y saber de forma inmediata los usuarios morosos o cualquier otra información relevante.

2.2. Objetivos del sistema

- Objetivo general: “Desarrollar una solución tecnológica para el edificio *El mirador* para gestionar las ganancias y gastos de este, asegurando un proceso transparente y mejorando la eficiencia de cobros y reparaciones”
- Objetivos específicos :
 - Registrar la información de los usuarios en una base de datos
 - Realizar cálculos y registros de pagos
 - Almacenar las solicitudes y mantenciones del edificio
 - Generar reportes relacionados a deudas o pagos

2.3. Requerimientos funcionales

1. Gestionar el personal: el sistema debe gestionar de manera eficiente todo usuario relacionado al edificio, incluyendo datos como nombre, contacto y/o número de departamento.
2. Gestionar los pagos: el sistema se encargará de almacenar los registros de los pagos y realizar los cálculos necesarios para realizar todas las operaciones de relevancia.
3. Consulta de deudas: el sistema le dará acceso a los administradores para visualizar a usuarios que presenten deudas pendientes.
4. Registro de solicitudes: el sistema debe permitir ingresar a los usuarios las solicitudes o reclamos en relación al edificio.
5. Consulta de estados de solicitudes: el sistema debe permitir a los usuarios realizar consultas con respecto a las solicitudes ingresadas.
6. Gestión de mantenimiento: el sistema debe llevar un registro sobre las tareas de mantenimiento, marcadas como realizadas o pendientes.
7. Notificaciones a usuarios: el sistema debe notificar de manera oportuna

sobre deudas pendientes o información relevante del edificio.

8. Generación de reportes: el sistema debe generar reportes en base a pagos, deudas, solicitudes y mantenimiento del edificio para el administrador.

2.4. Requerimientos no funcionales

1. El sistema debe registrar de manera segura la información de los usuarios.
2. El sistema debe permitir iniciar sesión a los usuarios.
3. El sistema debe restringir el acceso acorde a los roles del edificio.
4. El sistema debe cerrar sesión de usuarios tras periodo de inactividad.
5. El sistema debe responder a las solicitudes en menos de 2 segundos.
6. El sistema debe soportar a múltiples usuarios simultáneamente.
7. El sistema debe tener una interfaz amigable e intuitiva, fácil de usar.
8. El sistema debe ser de tipo responsive.
9. El sistema debe estar disponible el 99% del tiempo
10. El código del sistema debe permitir modificaciones sin afectar otras funcionalidades.
11. El sistema debe evitar duplicidad de datos y un correcto ingreso de estos.
12. El sistema debe ser compatible con navegadores modernos.
13. El sistema debe ser compatible con diferentes sistemas operativos.
14. El sistema debe enviar notificaciones oportunas.
15. El sistema debe realizar copias de seguridad periódicamente.
16. El sistema debe permitir recuperar datos ante fallos.

2.5. Supuestos y dependencias

- Supuestos: Asumimos que los vecinos tienen acceso a un teléfono o computador con internet para poder meterse al sistema.
- Dependencias: El sistema va a depender de una integración con alguna pasarela de pago externa (como Mercado Pago o Webpay) para poder

procesar las transacciones de forma automática y segura.

3. ESTILOS Y PATRONES ARQUITECTÓNICOS

3.1. Estilo arquitectónico adoptado (ej. monolítico, microservicios, SOA, capas)

3.2. Justificación del estilo según el contexto del sistema

3.3. Patrones de diseño aplicados (ej. patrón MVC, repositorio, etc.)

4. MODELO 4 +1 Y VISTAS ARQUITECTÓNICAS

4.1. VISTA DE ESCENARIO

4.1.1. Propósito

4.1.2. Actores

4.1.3. Diagrama general de casos de uso

4.1.4. Diagrama de casos de uso específicos

4.1.5. Lista de casos de uso

Código	Nombre	Actores
CU-001-001	Exportar saldos y puntos a vencer	
CU-002-001	Exportar actividades	
CU-002-002	Importar deuda vencida por PDV	
CU-004-001	Generación Archivo PDA Importación	

4.1.6. Especificación de casos de uso

Caso de Uso	[Nombre del Caso de Uso]	Identificador: [Del caso de uso]
Actores	[Listado de los actores que tienen participación en el caso de uso]	
Tipo	[Tipo de caso de uso, primario, secundario, opcional]	
Referencias	[Requerimientos o funcionalidades incluidas en este caso de uso. Casos de uso relacionados.]	
Precondición	[Condiciones sobre el estado del sistema que deben cumplirse para iniciar el caso de uso]	
Postcondición	[Efectos inmediatos que tienen la ejecución del caso de uso sobre el estado del sistema]	

Descripción	[Descripción del caso de uso]
Resumen	[Resumen de alto nivel del funcionamiento]

CURSO NORMAL

Nro.	Ejecutor	Paso o Actividad
[Nro. de paso]	[Actor ejecutor o especifica si es el sistema o subsistema]	[Descripción del paso actividad ejecutado]
[Se describe el proceso o secuencia de pasos ejecutadas usando frases cortas] [Cada paso del proceso puede ser ejecutado por los Actores o por el sistema] [Se describe la secuencia de acciones realizadas por los actores y la secuencia de actividades realizada por el sistema como respuesta].		

CURSO ALTERNATIVO

Nro.	Descripción de acciones alternas
[Número o de paso]	[Descripción de la secuencia de acciones alternas para el número de actividad indicado. Debe hacer referencia al número de paso en el curso normal]
[Cada paso descrito en el curso normal, puede tener actividades alternas, según la distribución de escenarios que ocurra en el flujo de procesos, en esta ficha se completa para cada actividad (haciendo referencia a su número) las posibles secuencias alternas]	

4.2. VISTA LÓGICA

4.2.1. Propósito

4.2.2. Diagrama de clases

4.2.3. Descripción diagrama de clases

4.3. VISTA DE IMPLEMENTACIÓN/DESARROLLO

- 4.3.1. Propósito
- 4.3.2. Diagrama de componente
- 4.3.3. Descripción diagrama de componente
- 4.3.4. Diagrama de paquete
- 4.3.5. Descripción diagrama de paquete

4.4. VISTA DE PROCESOS

- 4.4.1. Propósito
- 4.4.2. Diagrama de actividad
- 4.4.3. Descripción diagrama de actividad

4.5. VISTA FÍSICA

- 4.5.1. Propósito
- 4.5.2. Diagrama de despliegue
- 4.5.3. Descripción diagrama de despliegue

5. REQUISITOS DE CALIDAD

5.1. Propósito

5.2. Atributos de calidad (por ejemplo: Usabilidad, Accesibilidad (WCAG), Rendimiento, Mantenibilidad, Seguridad Portabilidad)

ATRIBUTO DE CALIDAD	DESCRIPCION	JUSTIFICACIÓN

5.3. Reglas y criterios de evaluación de calidad

Cómo se medirá el cumplimiento de cada atributo (ej. tiempo de carga < 2 seg, puntuación Nielsen de usabilidad, cumplimiento WCAG nivel AA, etc.).

Herramientas o métodos que se utilizarán(pruebas de carga, pruebas heurísticas, inspecciones, validación con usuarios, etc)

6. PRINCIPIOS DE DISEÑO APLICADOS

6.1. Propósito

6.2. Principios de diseño (por ejemplo: abstracción, acoplamiento, cohesión, encapsulamiento, modularidad)

PRINCIPIO	DESCRIPCIÓN	APLICACIÓN EN EL SISTEMA
Cohesión	Cada módulo o clase tiene una única responsabilidad bien definida.	Los servicios están diseñados para realizar tareas específicas y no múltiples funciones

7. PROTOTIPO

7.1. Propósito

7.2. Mockups (imágenes con una breve descripción)

7.3. Justificar herramientas de prototipado

8. EVALUACIÓN DE CALIDAD HEURÍSTICA DE NIELSEN

8.1. Propósito

8.2. Lista de verificación

8.3. Análisis y métricas de resultados

9. CONTROL DE VERSIONES

9.1. Propósito

9.2. Control de versión utilizado (justificar el tipo de control de versión utilizado (fecha, semántica o secuencial)

9.3. Justificar herramientas de versionamiento

7. CONCLUSIONES

8. BIBLIOGRAFÍA

9. ANEXOS

9.1. Carta Gantt